



POLITECNICO
MILANO 1863

Continual Learning

Federico Giannini

Ph.D. Student

DEIB – Politecnico di Milano

federico.giannini@polimi.it

- ▶ The Traditional ML approach is unrealistic:
 - ▶ It supposes that the knowledge is completely represented by a bounded training set.
 - ▶ It supposes that data will not change its distribution over time.
- ▶ In the real world, data:
 - ▶ Becomes available over time.
 - ▶ New distributions and new problems may appear over time.
- ▶ We cannot train a model once on a data set and think it will remain appropriate forever.
- ▶ We must continuously train our model to keep it updated with new knowledge.

Introduction

Suppose that we want to distinguish between odd and even hand-written digits

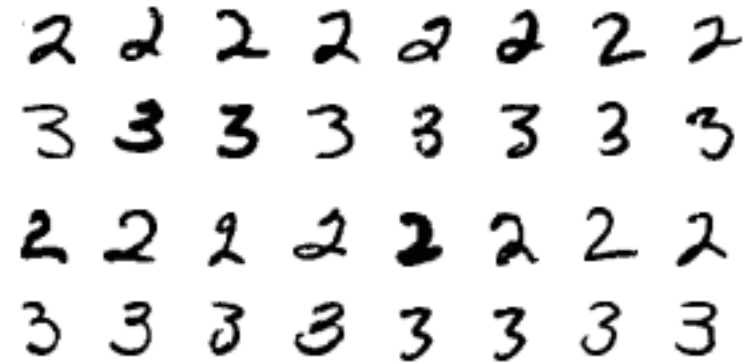
Original distribution



The model initially observes 0s and 1s.



New distribution



The drift introduces the new digits 2s and 3s, which do not invalidate what the model has learned about 0s and 1s.

It just extends the problem to new instances not yet observed.

The goal is to learn how to classify 2s and 3s without forgetting what the model learned when trained on 0s and 1s.

Introduction

Suppose now that we want to identify objects within images.

Original distribution



The model initially observes images of snow landscapes.

DRIFT
➔

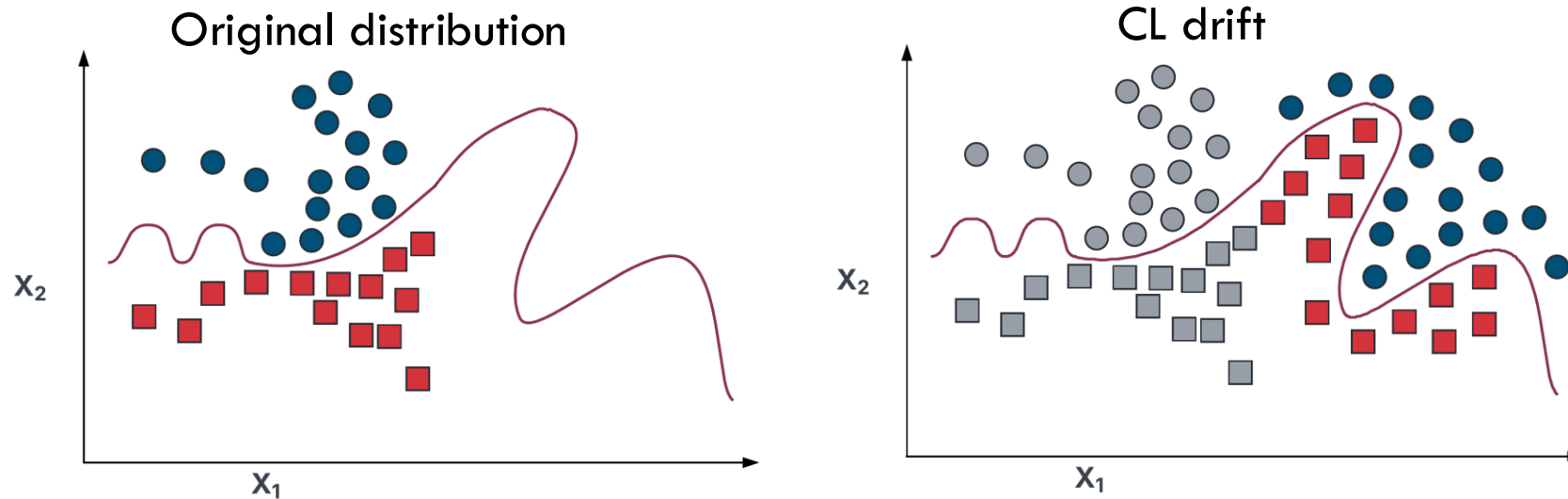
New distribution



The drift introduces sea landscapes, which do not invalidate what has been learned about snow landscapes. It just extends the problem to new instances not yet observed. It may also add new types of objects.

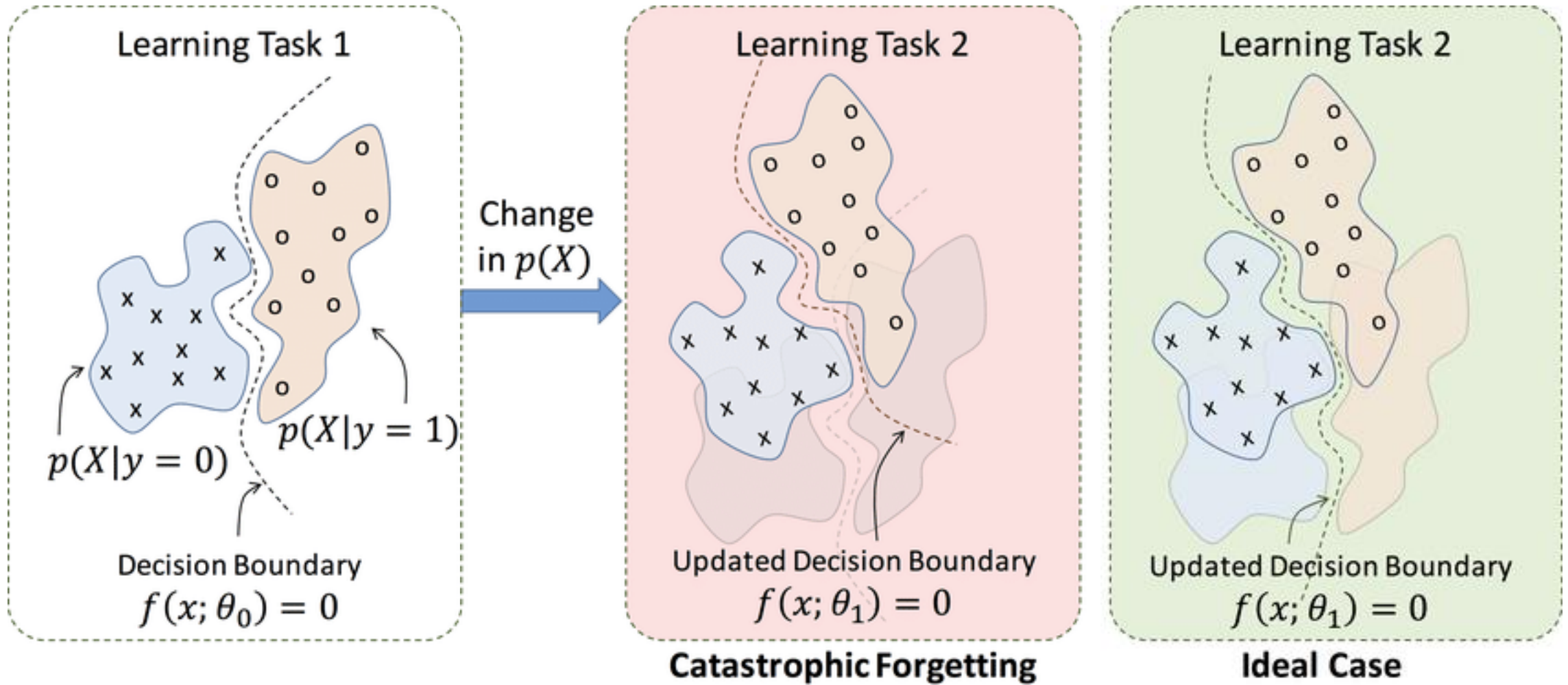
The goal is to learn how to identify objects in sea landscapes without forgetting snow landscapes.

Drifts in CL



- CL assumes that a drift introduces a new input data distribution!
- What the model learned in the past remains valid. The new subproblem does not contradict the previous ones.
- We do not want to lose previous knowledge when incorporating new one.

Catastrophic Forgetting



Stability-plasticity Dilemma

A model should achieve a trade-off between stability and plasticity.

- ▶ **Plasticity**: ability to acquire new knowledge and keep learning over time.

Too much plasticity → forgetting past knowledge.

- ▶ **Stability**: ability to remember the past acquired knowledge over time.

Too much **stability** → difficulties in learning new knowledge.

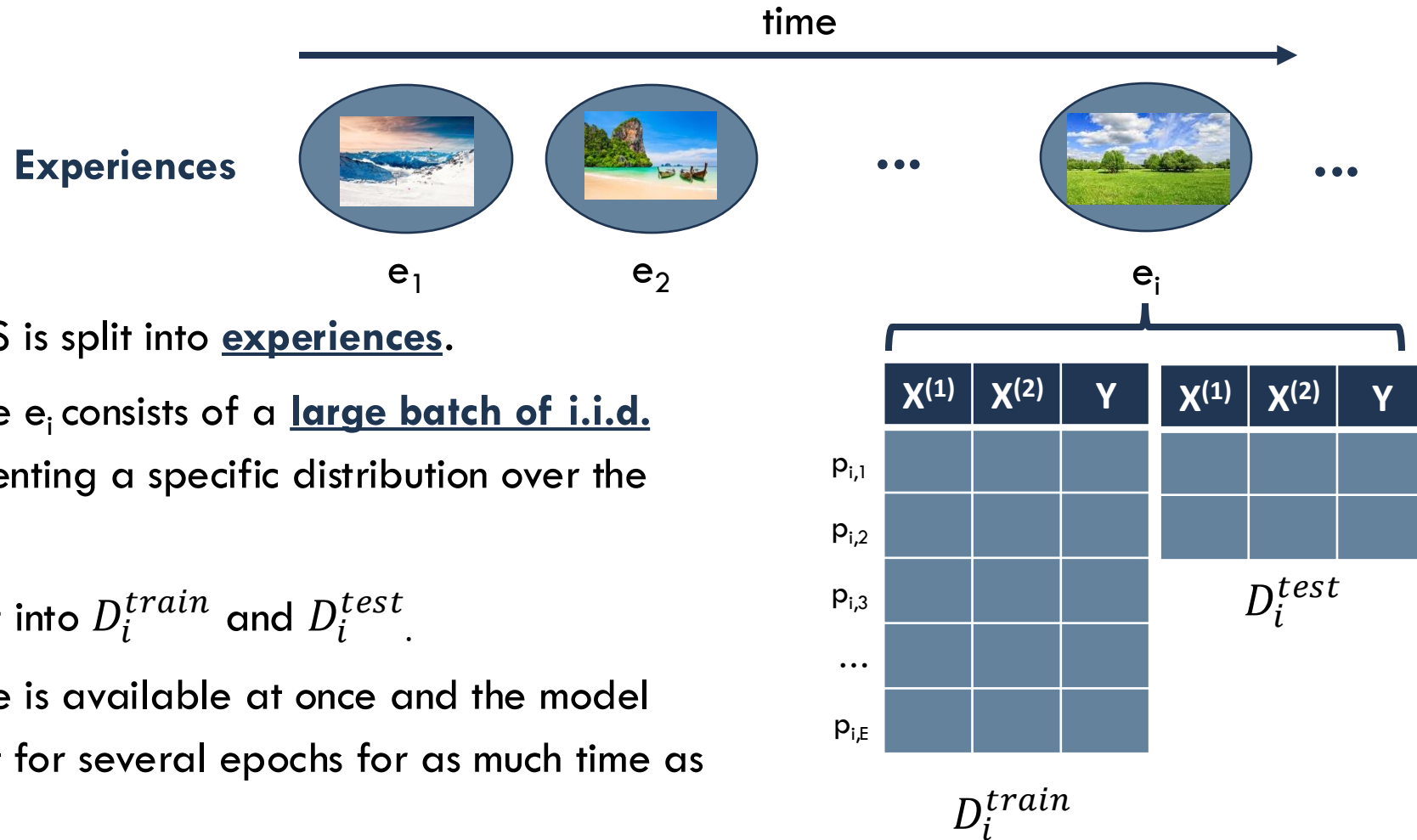
CL focuses on the following:

- ▶ Learning new knowledge.
- ▶ Remembering past knowledge.
- ▶ Generalizing on future instances.

Evaluation strategies and different methodologies are associated with these goals.



CL Problem Setting



- ▶ A data stream S is split into experiences.
- ▶ Each experience e_i consists of a large batch of i.i.d. samples D_i , representing a specific distribution over the feature space X .
- ▶ D_i is usually split into D_i^{train} and D_i^{test} .
- ▶ Each experience is available at once and the model can be trained on it for several epochs for as much time as needed.

$$A^{CL}: (f_{i-1}^{CL}, M_{i-1}, D_i^{train}, t_i) \rightarrow (f_i^{CL}, M_i)$$

- ▶ f_i^{CL} : model learned after the i-th experience.
- ▶ M_i : knowledge until the i-th experience.
- ▶ D_i^{train} : training set associated with the i-th experience.
- ▶ t_i : **task** (optional). It could be seen as a time interval in which the data distribution and the objective function may be fixed (similar to SML's concept).

The term “task” refers to the classification problem, “experience” refers to the associated data.

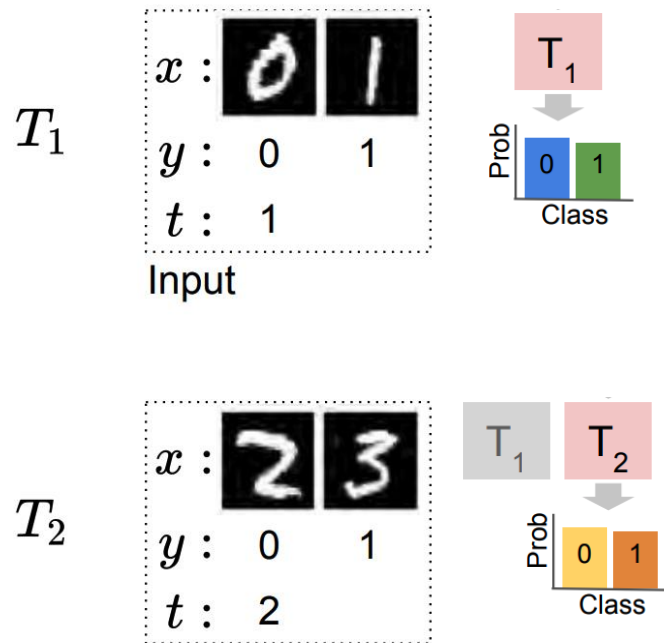
A CL strategy must perform well on the current experience and all the experiences seen so far.

Scenarios

A new experience usually introduces a change. (Each experience is supposed to be i.i.d.)

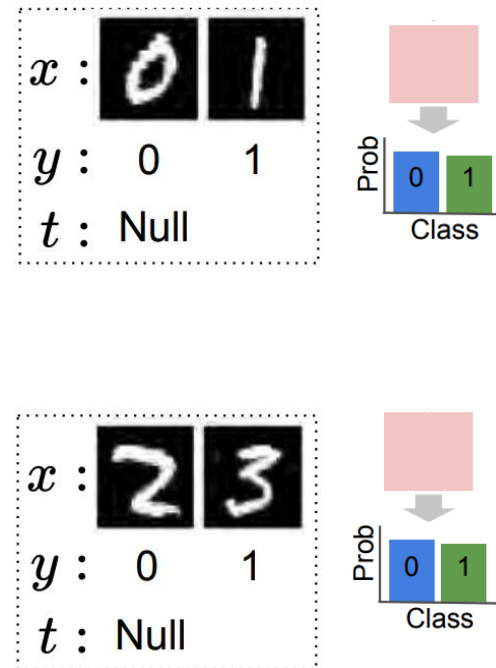
→ **No need to apply change detectors during training.**

Incremental Task Learning



Task labels during train & test.

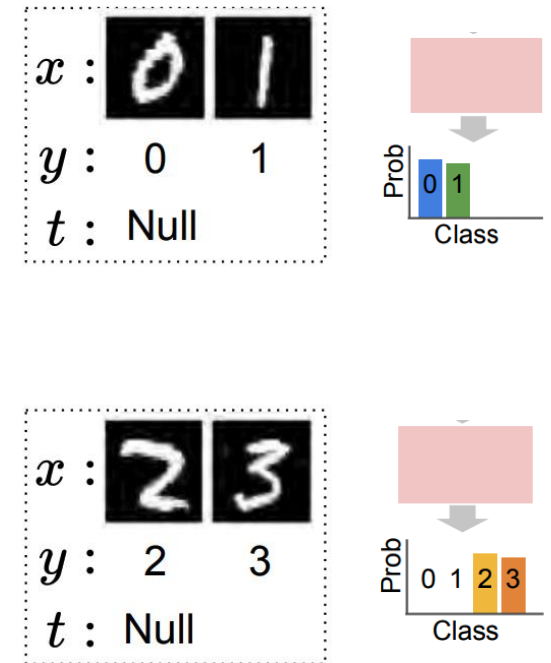
Incremental Domain Learning



No task labels.

Each experience has its own class labels.

Incremental Class Learning



No task label

(different class labels during train)

Scenarios

	Training	Testing
Incr. Domain	$\{\mathcal{D}_t\}_{t \in \mathcal{T}}; p(\mathcal{X}_i) \neq p(\mathcal{X}_j) \text{ and } \mathcal{Y}_i = \mathcal{Y}_j \text{ for } i \neq j$	$\{p(\mathcal{X}_t)\}_{t \in \mathcal{T}}, t \text{ is not required}$
Incr. Task	$\{\mathcal{D}_t, t\}_{t \in \mathcal{T}}; p(\mathcal{X}_i) \neq p(\mathcal{X}_j)$	$\{p(\mathcal{X}_t)\}_{t \in \mathcal{T}}; t \text{ is available}$
Incr. Class	$\{\mathcal{D}_t\}_{t \in \mathcal{T}}; p(\mathcal{X}_i) \neq p(\mathcal{X}_j) \text{ and } \mathcal{Y}_i \cap \mathcal{Y}_j = \emptyset \text{ for } i \neq j$	$\{p(\mathcal{X}_t)\}_{t \in \mathcal{T}}; t \text{ is unavailable}$

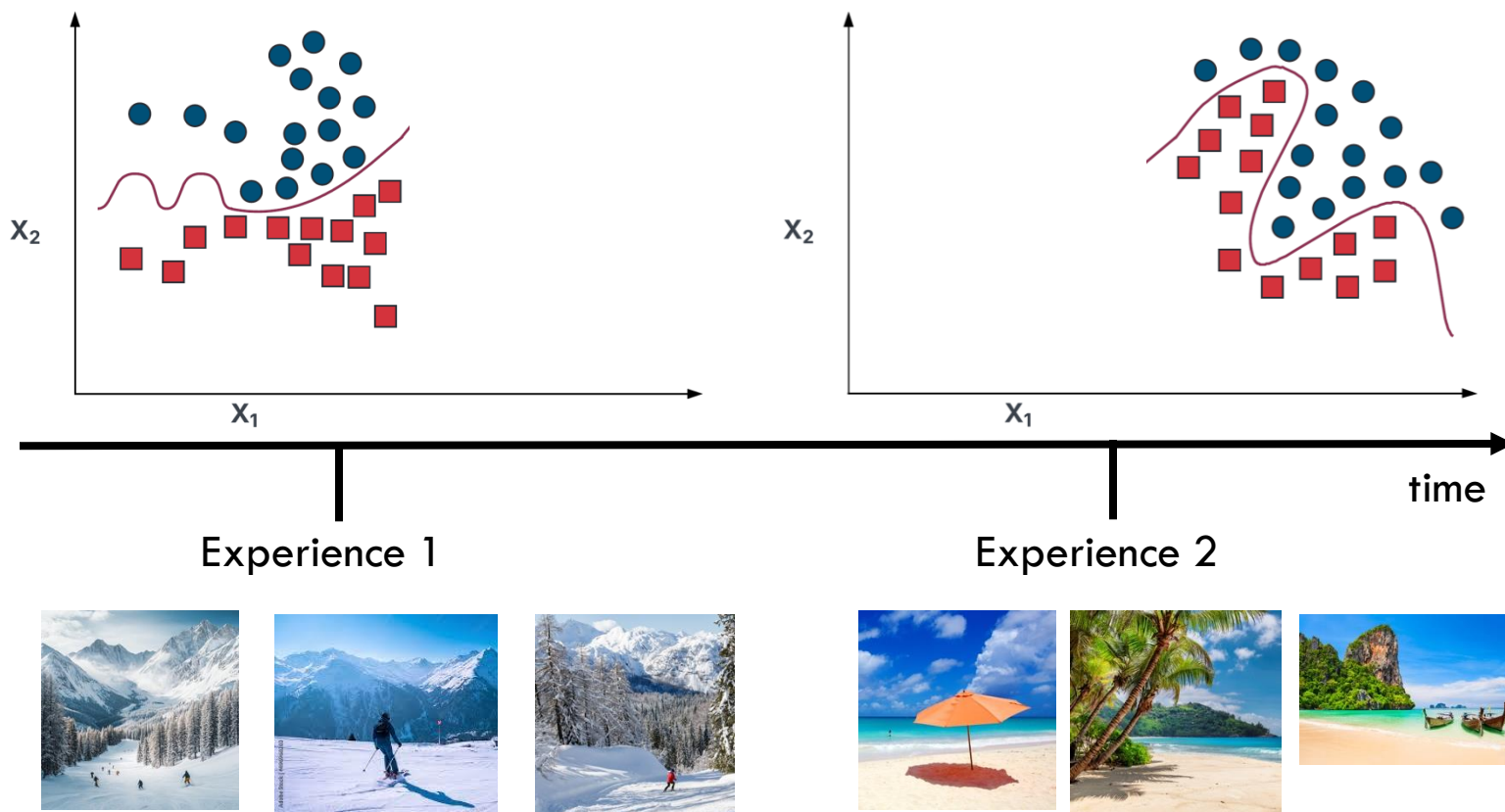
- t : identifier of a task (experience)
- $\mathcal{T}$: set of task identifiers
- $\mathcal{D}_t$: training set of task t
- $\mathcal{X}_t$: input data of task t
- $\mathcal{Y}_t$: set of target labels of task t

CL's Assumptions

- Each new task introduces a new input data distribution with no overlap with previous.
- One can imagine a general and complex problem split into subproblems, each defined in a specific input distribution.
- An experience contains data points associated with a specific subproblem.
- A concept drift introduces a new experience, representing a new subproblem that extends the original problem without contradicting the previous ones, as it has no overlap with them.



What is a CL experience?



We can imagine accumulating the data points of each concept in a batch (experience).

When the concept finishes (an abrupt concept drift arises) we input the experience to the CL model.

Each new experience can introduce:

- New Instances of the same classes in a new feature subspace (**Domain Incremental**).
- New classes (**Class Incremental**)
- We have the label of the distribution in **Task Incremental**.

CL Evaluation Protocol

	Test e_1	Test e_2	Test e_3
Train e_1	$R_{1,1}$	$R_{1,2}$	$R_{1,3}$
Train e_2	$R_{2,1}$	$R_{2,2}$	$R_{2,3}$
Train e_3	$R_{3,1}$	$R_{3,2}$	$R_{3,3}$

Each experience is split into train and test sets.

After each experience's training, CL computes the metrics on **all** the experiences' test sets (assuming all test sets are available in advance).



What learning abilities should we measure?

- **Plasticity**: the model can learn the current experience (it obtains good performance on the current experience's test set).
- **Stability**: the model does not lose performance on the previous experiences. If it does, the model suffers from catastrophic forgetting.
- **Backward transfer**: when learning the current experience, the model increases the performance on the previous ones. Backward transfer implies, of course, stability.
- **Forward transfer**: when learning the current experience, the model learns also the next experience (it outperforms the random model on the next experience).

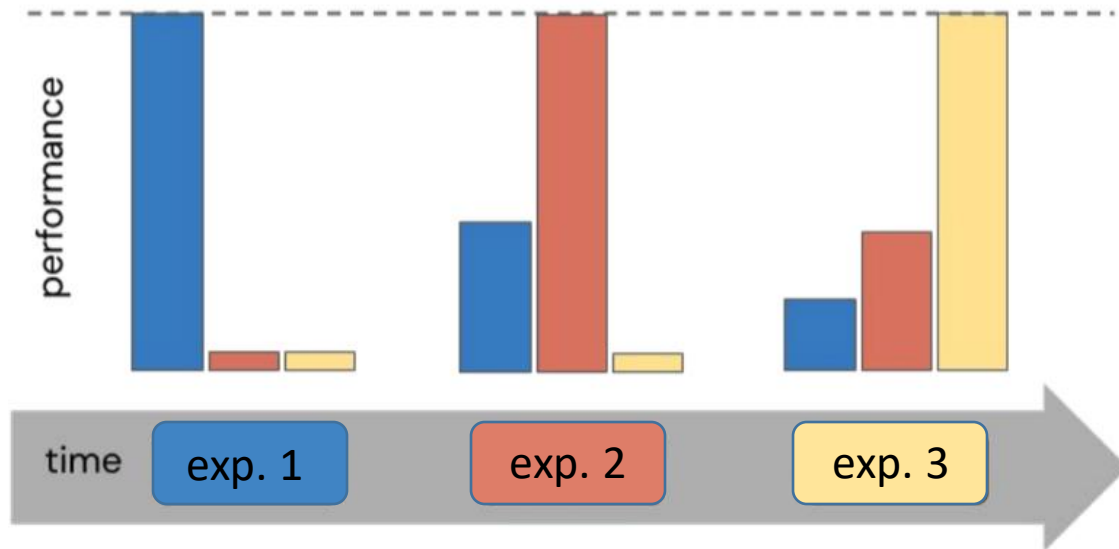


Stability-plasticity Dilemma: Situations to avoid

Situations to avoid:

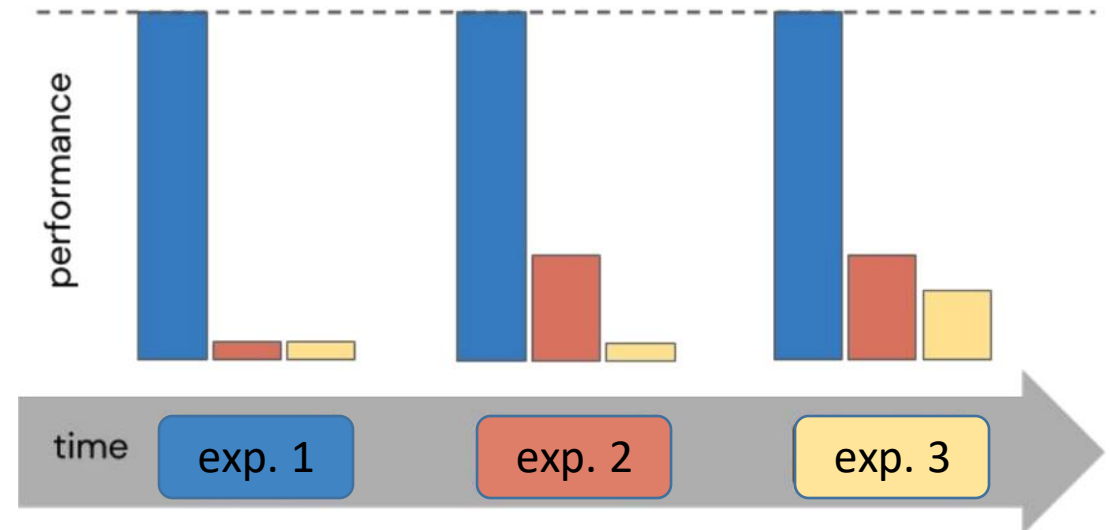
Catastrophic Forgetting

Training on new experiences reduces performance on previously learned experiences.



Model with no Plasticity

The model does not keep learning as new experiences are observed.



CL Evaluation: Final Accuracy

	Test e ₁	Test e ₂	Test e ₃
Train e ₁	$R_{1,1}$	$R_{1,2}$	$R_{1,3}$
Train e ₂	$R_{2,1}$	$R_{2,2}$	$R_{2,3}$
Train e ₃	$R_{3,1}$	$R_{3,2}$	$R_{3,3}$

Final Accuracy:

$$ACC_{final} = \frac{1}{T} \sum_{i=1}^T R_{T,i}$$

Mean of final performances on all the experiences' test sets.

It measures how the final model performs on all the experiences.

CL Evaluation: Average Accuracy over time

	Test e ₁	Test e ₂	Test e ₃
Train e ₁	R _{1,1}	R _{1,2}	R _{1,3}
Train e ₂	R _{2,1}	R _{2,2}	R _{2,3}
Train e ₃	R _{3,1}	R _{3,2}	R _{3,3}

Average Accuracy over time:

$$ACC_{avg} = \frac{\sum_{i=1}^N \sum_{j=1}^i R_{i,j}}{\frac{N(N+1)}{2}}$$

After training on each experience, it tests on the current and the previous experiences. Finally, it averages all the values.

It measures how the model performs over time on the current and previous experiences.

CL Evaluation: Final Backward Transfer

	Test e ₁	Test e ₂	Test e ₃
Train e ₁	<u>R_{1,1}</u>	R _{1,2}	R _{1,3}
Train e ₂	R _{2,1}	<u>R_{2,2}</u>	R _{2,3}
Train e ₃	R _{3,1}	R _{3,2}	R _{3,3}

Final Backward Transfer:

$$BWT_{final} = \frac{1}{T-1} \sum_{i=1}^{T-1} R_{T,i} - R_{i,i}$$

It measures how the final model has improved or decreased the performances on the previous experiences.

BWT<0 indicates forgetting!

BWT>=0 indicates stability!

CL Evaluation: Average Backward Transfer over time

	Test e ₁	Test e ₂	Test e ₃
Train e ₁	<u>R_{1,1}</u>	R _{1,2}	R _{1,3}
Train e ₂	R _{2,1}	<u>R_{2,2}</u>	R _{2,3}
Train e ₃	R _{3,1}	R _{3,2}	R _{3,3}

Average Backward Transfer over time:

$$BWT_{avg} = \frac{\sum_{i=2}^T \sum_{j=1}^{i-1} (R_{i,j} - R_{j,j})}{\frac{T(T-1)}{2}}$$

After each training, it measures how the current model has improved or decreased the performances on the previous experiences. Finally, it averages all the performance.

BWT<0 indicates forgetting!

BWT>=0 indicates stability!

CL Evaluation: Forward Transfer

	Test e ₁	Test e ₂	Test e ₃
Train e ₁	$R_{1,1}$	$R_{1,2}$	$R_{1,3}$
Train e ₂	$R_{2,1}$	$R_{2,2}$	$R_{2,3}$
Train e ₃	$R_{3,1}$	$R_{3,2}$	$R_{3,3}$

Forward Transfer:

$$FWT = \frac{1}{T-1} \sum_{i=2}^T R_{i-1,i} - \bar{b}_i$$

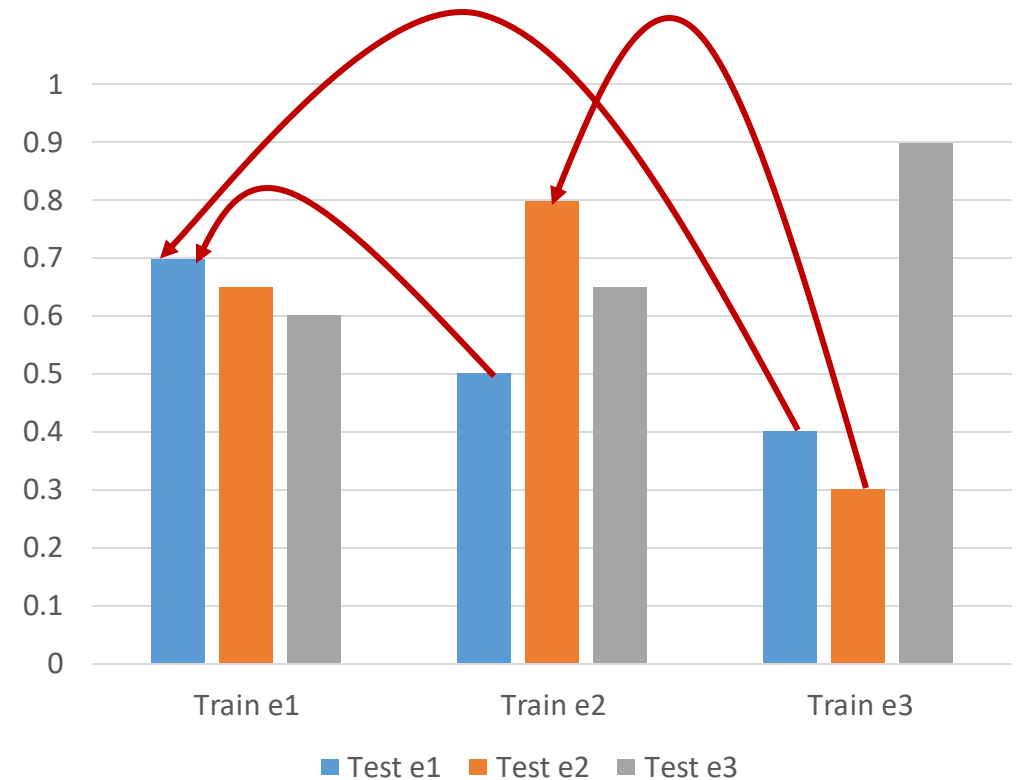
It measures how the current training improves performance for the next experience.

$\bar{b}_i$ is the performance obtained by the random model on the test set of the experience i .

After training on each experience, it compares the performance obtained by the random model on the next experience with the one obtained by the model.

CL Evaluation Example

	Test e_1	Test e_2	Test e_3
Train e_1	0.7	0.65	0.6
Train e_2	0.5	0.8	0.65
Train e_3	0.4	0.3	0.9

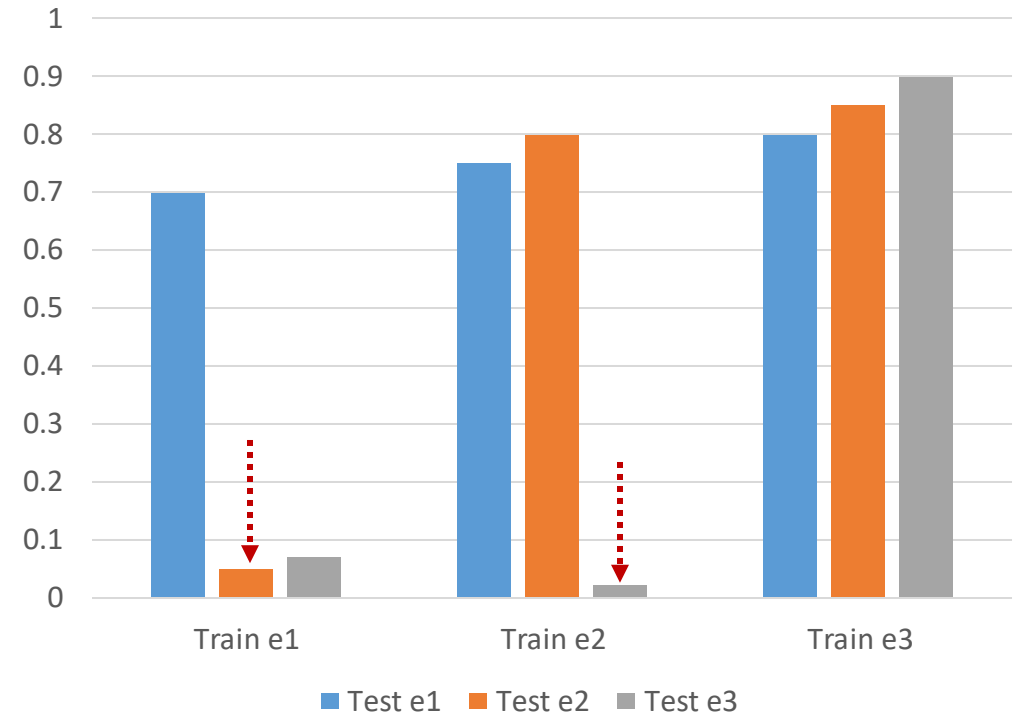


Over time , the model loses performance on previously learned experiences.

Negative Backward Transfer (and thus catastrophic forgetting)

CL Evaluation Example

	Test e_1	Test e_2	Test e_3
Train e_1	0.7	0.05	0.07
Train e_2	0.75	0.8	0.02
Train e_3	0.8	0.85	0.9



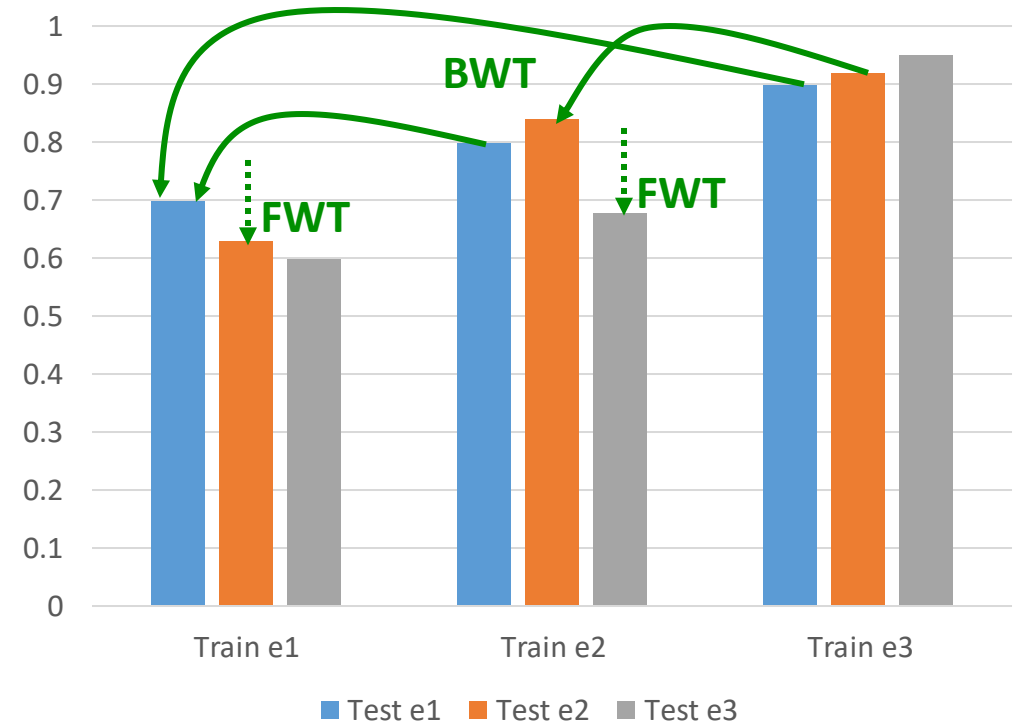
Suppose that we have **5 classes** for each experience. The model performs worse than random guessing on the next experience.

Negative Forward Transfer



CL Evaluation Example

	Test e_1	Test e_2	Test e_3
Train e_1	0.7	0.63	0.6
Train e_2	0.8	0.84	0.68
Train e_3	0.9	0.92	0.95

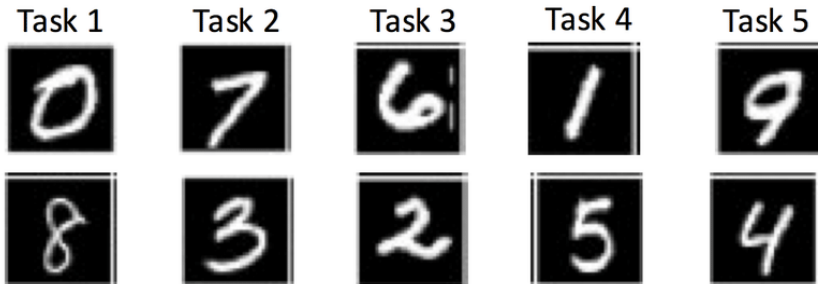


The model improves performance over time when tested on previous experiences. **Positive Backward Transfer (and thus stability).**

Suppose that we have **5 classes** for each experience. The model performs better than random guessing on the next experience. **Positive Forward Transfer.**

CL Applications (toy dataset): MNIST and CIFAR

MNIST



These datasets are typically used in the **Incremental Class Learning** scenario, where each experience contains new instances associated with new classes. However, they can also be applied to the Incremental Task Scenario.

MNIST may also be applied to an Incremental Domain Scenario where each task contains odd vs even.

CIFAR



CL Applications (realistic datasets): CLEAR



The labels keep the same.

Each experience introduces images from a different year.

This setting fits an

Incremental Domain Scenario.

The model must remember objects observed during previous years.

This is also an interesting case for investigating Forward Transfer.

- ▶ CL aims to learn new tasks without losing the prediction ability on old ones.
- ▶ CL strategies are meant with the specific goal of being plastic and reducing as much as possible forgetting. Obtaining $BWT \geq 0$ is the theoretical optimal case, which is often difficult to achieve in practice.
- ▶ FWT is not a specific goal of CL strategies, but could be a nice-to-have positive side effect.
- ▶ A CL strategy is applied on top of a base learner.
- ▶ Each Neural Network model can be used as a base learner.

- ▶ **Replay:** during each experience, they mix old examples with new ones to not forget old tasks. Generative replay strategies generate examples associated with old tasks. The training set's size increases with the number of tasks.
- ▶ **Regularization:** they change the loss function by adding regularization terms to the new task's problem and not losing the old ones.
- ▶ **Architectural:** they introduce new parameters to the network (e.g. shared layers between all tasks and tasks-specific layers, transfer learning or weight masks). For some strategies, the number of parameters could increase with the number of tasks.

Conclusion

- ▶ CL operates in an environment where, at each timestamp, it receives a new large batch of data, representing new information that must be incorporated into the model without forgetting the previously acquired knowledge.
- ▶ In this context, drifts are assumed to affect just the input distribution. Each experience introduces a new subproblem (task) that has no overlap with the previous ones and does not contradict them.
- ▶ The crucial problem is to find a balance between plasticity and stability.
- ▶ CL evaluation distinguishes between train and test sets and measures different learning abilities (plasticity, backward transfer, forward transfer).



Thank you!



If you want to learn more about Continual Learning, watch the [Continual Learning Course](#) by Continual AI.